CIS 613:
Mutation Testing

1

Test Quality

How do we tell if tests are good?

- Creating them via decision tables, boundary value analysis, or equivalence classes.
- Code or path coverage metrics.
- How many failures they have already caught.
- How many failures they will catch.

Anything wrong with measuring how many failures a test *will* catch?

2

What is Mutation Testing?

- The purpose of testing is looking for errors.
- Assume all your tests pass.
- How do you tell if your tests are any good?

Would your tests catch errors, if the errors were there?

3

What is Mutation Testing?

- **Competent Programmer Hypothesis**: A programmer writes a program that is in the general neighborhood of the set of correct programs.
- **Coupling Effect**: Test data that distinguishes all programs differing from a correct one by only simple errors is so sensitive that it also implicitly distinguishes more complex errors.

Mutation testing alters programs by a little bit to see if test data will catch the error, assuming the alteration is similar to a bug.

4

Definitions

- **Mutation**: Syntactic change in a program
- **Mutation Analysis**: Assessing the quality of a test suite
- **Mutation Testing**: Improving the test suite using mutants
- **Mutant**: Change in a program to cause a fault
- **Killed Mutants (Dead Mutants)**: Mutants detected by test suite
- **Live Mutants**: Mutants not detected by the test suite

Mutation Score = Killed Mutants / Total Mutants

5

Simple Example

```
public static int simpleExample(int x, int y) {
   if(x < y)
      return x + y;
   else
      return x * y;
}

assertEquals(15,
   SimpleExample.simpleExample(5, 10));
```

6

Simple Example **Mutation**!

```
public static int simpleExample(int x, int y) {
   if(x < y)
      return x + y;
   else
      return x * y;
}
```

Operator Mutation

```
assertEquals(15,
   SimpleExample.simpleExample(5, 10));
```

7

Simple Example **Mutation**!

```
public static int simpleExample(int x, int y) {
   if(x < y)
      return x - y;
   else
      return x * y;
}
```

Operator Mutation

```
assertEquals(15,
   SimpleExample.simpleExample(5, 10));
```

Does our test still pass?
What does this say about our test?

8

What is the problem?

COST!

9

Solution?

Tool Support!

Java: http://pitest.org

Python: https://github.com/boxed/mutmut

10

```
public static int triangle_identifier(int a, int b, int c) {

	boolean isATriangle;

	// Is A Triangle?
	if((a < b + c) && (b < a + c) && (c < a + b))
		isATriangle = true;
	else
		isATriangle = false;

	// Determine Triangle Type
	int triangleType = INVALID;
	if(isATriangle) {
		if((a == b) && (b == c))
			triangleType = EQUILATERAL;
		else if((a != b) && (a != c) && (b != c))
			triangleType = SCALENE;
		else
			triangleType = ISOSCELES;
	} else
		triangleType = INVALID;

	return triangleType;
}
```

11

Example Tests: Run Normally

*assertEquals(Triangle.**INVALID**,*
Triangle.*triangle_identifier(0, 0, 0));*

*assertEquals(Triangle.**INVALID**,*
Triangle.*triangle_identifier(1, 1, 3));*

*assertEquals(Triangle.**EQUILATERAL**,*
Triangle.*triangle_identifier(2, 2, 2));*

*assertEquals(Triangle.**ISOSCELES**,*
Triangle.*triangle_identifier(2, 2, 3));*

*assertEquals(Triangle.**SCALENE**,*
Triangle.*triangle_identifier(2, 3, 4));*

But are our tests any good?

12

```
def triangle_identifier(a, b, c):

    is_a_triangle = None

    # Is A Triangle?
    if (a < b + c) and (b < a + c) and (c < a + b):
        is_a_triangle = True
    else:
        is_a_triangle = False

    # Determine Triangle Type
    triangle_type = INVALID
    if is_a_triangle:
        if (a == b) and (b == c):
            triangle_type = EQUILATERAL
        elif (a != b) and (a != c) and (b != c):
            triangle_type = SCALENE
        else:
            triangle_type = ISOSCELES
    else:
        triangle_type = INVALID

    return triangle_type
```

13

Example Tests: Run Normally

```
self.assertEquals(triangle.INVALID,
    triangle.triangle_identifier(0, 0, 0))

self.assertEquals(triangle.INVALID,
    triangle.triangle_identifier(1, 1, 3))

self.assertEquals(triangle.EQUILATERAL,
    triangle.triangle_identifier(2, 2, 2))

self.assertEquals(triangle.ISOSCELES,
    triangle.triangle_identifier(2, 2, 3))

self.assertEquals(triangle.SCALENE,
    triangle.triangle_identifier(2, 3, 4))
```

But are our tests any good?

14

So, what now?

Assignment!

15